Continuous Integration

We have tests. The next problem is making sure they get run.

A developer can make a change, try the part of the program that changed, and forget to run the rest of the tests. Another developer may have a different Python environment. A test that works on one machine may depend on a file that nobody remembered to commit. None of this requires anyone to be careless. There are simply a lot of details to remember.

Continuous integration (CI) combines frequent integration of changes with automated checks of the shared code. Small changes and prompt feedback make it easier to discover when work no longer fits together. A workflow helps with those checks; installing a workflow does not, by itself, establish the team's habit of integrating often. (1)

For this exercise, we will take an app that runs on our own machine, test it locally, and arrange for GitHub to run the same three test layers after each push. Then we will deliberately make a check fail and repair it. Branch rules and approval requirements come later. First we need to understand the evidence those rules will use.

The WebCalc application

The supplied `webcalc/` folder contains a small calculator implemented with NiceGUI. `calculator.py` holds the arithmetic and calculator state. `main.py` connects that logic to buttons and a display. It also accepts keyboard input.

This is a basic calculator: `2 + 3 * 4` gives `20`, because it evaluates the addition when the multiplication button is pressed. It does not apply algebraic operator precedence. That is a requirement to understand, not a defect to fix.

We use three different testing methods, each reaching a different part of the app. Direct calls exercise the calculator with pytest. NiceGUI simulates interaction with its Python elements. Finally, behave turns readable scenarios into WebDriver actions against the running app. Continuous integration will run all three methods, without requiring them to use the same testing tool.

The supplied tests are:

Layer	Supplied tests	What gets exercised	Expected result
Unit	`test_calculator.py`	Calculator methods and displayed values	27 tests pass
Simulated graphical user interface (GUI)	`test_main.py`	NiceGUI button handlers and display updates	12 tests pass
Browser	`features/calculator.feature`	The running app through Chrome and WebDriver	12 scenarios, 51 steps pass

The unit tests do not need NiceGUI. The simulated tests load the actual page but exercise its Python elements without a browser. NiceGUI provides this simulation mechanism for testing interface behavior. (2) The browser tests use Selenium WebDriver to interact with rendered buttons and send keyboard events to a real Chrome instance. Headless Chrome runs without a visible window; it is still a browser. (3)

The layers overlap deliberately. An addition test at each level can help locate a failure. If the arithmetic test passes but the browser test fails, the arithmetic may be fine while a button, an event, or the running environment is wrong. We still need to read the failure rather than guess from the job name.

Testing reach makes compartmentalization useful

Compartmentalization separates responsibilities so we can work on one part of a program without involving everything else. Testing reach is our ability to exercise those parts directly and inspect what they do. The two belong together.

When people cannot easily reach a component with a test, they fall back on whatever interface they can operate. They may test the math by clicking buttons on the web screen. Soon, checking addition requires a running server, a browser, and working event handlers. A failure anywhere along that path can interrupt a test whose real purpose was to check arithmetic.

That experience can erode confidence in compartmentalization. The code has been divided into parts, but checking one part still requires the whole application. The separation is less useful in everyday work than it looked in the design.

WebCalc gives us direct access to each responsibility. Test the arithmetic by calling `Calculator`. Use NiceGUI's simulation to check that the buttons invoke the intended operations and update the display. Use WebDriver to check that the running application responds correctly in a real browser. We can cover many arithmetic cases without repeating all of them through the screen.

The browser checks still matter. They establish that the pieces work together along the paths we exercise. Their job is to add evidence about those connections, while the direct tests carry the detailed arithmetic checks. **Testing reach makes compartmentalization useful:** we can change and check a component on its own, then verify its connections to the rest of the app.

Prepare a separate repository folder

Extract the supplied `webcalc.zip` into a fresh working location. The archive contains a `webcalc/` folder ready to become a repository of the same name. It includes the hidden `.github` directory and `.gitignore`, but excludes Git history, virtual environments, caches, logs, and generated test reports. Private files are excluded too.

The extracted folder contains the app, tests, workflow, and three text guides:

```
webcalc/
  calculator.py
  main.py
  test_calculator.py
  test_main.py
  requirements-test.txt
  CONTINUOUS_INTEGRATION.txt
  CREATE_GITHUB_REPO.txt
  RUN_BROWSER_TESTS.txt
  run_browser_tests.py
  features/
    calculator.feature
    environment.py
    steps/calculator_steps.py
  .github/workflows/webcalc-tests.yml
  .gitignore
```

The workflow must be under `.github/workflows` at the new repository's root. GitHub looks there for workflow files. Leaving it nested inside a course chapter would not activate it in the course repository. (4)

Open a terminal in your extracted `webcalc` directory. The commands below run there. Follow this chapter for the exercise. `CREATE_GITHUB_REPO.txt` provides a standalone version of the setup instructions; if the two differ, follow this chapter.

Run everything locally first

Install Python 3.12 and Chrome. We use Python 3.12 here because the Actions jobs use it too. Newer Python may also work, but using the same minor version removes one source of differences between local and hosted results. Create a virtual environment so the exercise has its own Python packages.

On macOS or Linux:

```
python3.12 -m venv .venv
source .venv/bin/activate
```

On Windows PowerShell:

```
py -3.12 -m venv .venv
.venv\Scripts\Activate.ps1
```

After activation, the following commands use `python` on either platform:

```
python -m pip install -r requirements-test.txt
python main.py
```

Open `http://localhost:8080` in a browser and try the calculator. Enter `2 + 3 =`; the display should show `5`. Stop the app with Ctrl+C in its terminal before running the automated browser runner below.

Now run each test layer separately:

```
python -m pytest -v test_calculator.py
python -m pytest -v test_main.py
python run_browser_tests.py
```

Expect 27 unit tests, 12 simulated GUI tests, and 12 browser scenarios containing 51 steps to pass. The last command starts WebCalc, waits for it to respond, runs behave in headless Chrome, and stops the app afterward. It requires port 8080 to be free. Selenium Manager locates or obtains a matching ChromeDriver; its first run may need internet access. (3)

For a visible browser run, start `python main.py` in one terminal and run `python -m behave` in a second terminal in the same directory. That behave command uses the app you started. It does not stop it.

Do not move on to Actions with an unexplained local failure. Read the failing assertion or startup error first. At this stage we are establishing a known passing version, not asking GitHub to diagnose a broken setup for us.

Save the passing version in Git

Git records versions in your local repository. A commit records the staged files; a push sends commits to the remote repository. We only need a few commands for this exercise. Version Control and GitHub Workflow will use these established checks when introducing branches and review.

From the extracted `webcalc` folder:

```
git init -b main
git add .
git status --short
```

Inspect the staged files. The app, tests, and `.github/workflows/webcalc-tests.yml` should be present. `.venv`, caches, test reports, and private files should not be staged. The supplied `.gitignore` excludes those generated and private paths.

```
git commit -m "Add WebCalc with three layers of tests"
```

This is our passing baseline. The tests passed locally before this version goes to GitHub.

Create the webcalc repository and push

The command-line route uses the GitHub command-line interface (CLI), whose command is `gh`. Install it and sign in, then create a private repository named `webcalc` under the authenticated account and push the local commit. The `--source`, `--remote`, and `--push` options connect the current folder to the new repository and upload its commits. (5)

```
gh auth login
gh repo create webcalc --private --source=. --remote=origin --push
```

Use `--public` instead of `--private` when a public repository is intended. If your account already has a repository named `webcalc`, inspect it first. Do not overwrite it or force-push for this exercise. Use an account where the name is available, or intentionally connect to an existing empty repository.

There is also a website route. On GitHub, create a repository named `webcalc` and choose its visibility. Leave the README, license, and `.gitignore` initialization options unchecked. The local folder already has a first commit. Replace `YOUR_USERNAME` below with the account name:

```
git remote add origin https://github.com/YOUR_USERNAME/webcalc.git
git push -u origin main
```

Choose one creation route. Do not run both sets of remote-creation commands.

What GitHub Actions does

GitHub Actions runs a workflow in response to repository events. A workflow contains jobs; jobs contain steps. A GitHub-hosted runner provides the machine that executes a job. Our workflow is written in YAML, a text format for structured data. (4)

Open `.github/workflows/webcalc-tests.yml`. Its first section names the workflow and lists its triggers:

```
name: WebCalc tests

on:
  push:
  pull_request:
  workflow_dispatch:
```

A push starts a run. Pull request events also start runs. `workflow_dispatch` makes a manual Run workflow button available. We will use ordinary pushes for this first exercise.

The job uses a matrix to run three variations of the same setup. Each variation gets a different `layer` value. `fail-fast: false` allows the other variations to finish even when one fails. (6)

```
strategy:
  fail-fast: false
  matrix:
    layer: [unit, simulated-gui, browser]
```

Each job checks out the repository, sets up Python 3.12, and installs `requirements-test.txt`. A condition selects the command for that layer. For example:

```
- name: Test calculator logic
  if: matrix.layer == 'unit'
  run: python -m pytest -v test_calculator.py --junitxml=test-results/unit.xml
```

The extra pytest option writes a machine-readable report in JUnit Extensible Markup Language (XML) format. It does not change which tests run. The simulated GUI job uses the equivalent command for `test_main.py`. The browser job calls `run_browser_tests.py` with behave's report options.

After a push	Unit job	Simulated GUI job	Browser job
Prepare	Checkout, Python, packages	Checkout, Python, packages	Checkout, Python, packages
Exercise	Calculator methods	NiceGUI simulation	Start app; run Chrome scenarios
Report	pytest result	pytest result	behave result and app log

These are separate jobs, so one job does not depend on a development server left running by another. The browser job creates its own app process. The supplied workflow uses Chrome on the Ubuntu runner and a ten-minute job timeout. Report files are uploaded as artifacts, along with the browser log and any failure screenshots. An artifact is a saved file attached to a workflow run. (7)

Inspect the first passing run

Open the new repository's **Actions** tab. Select **WebCalc tests**, then the run associated with the first push. The three results should become green:

```
WebCalc / unit
WebCalc / simulated-gui
WebCalc / browser
```

Open each job and expand its test step. Confirm the expected counts rather than just noticing the green icon. If a job fails, read its setup and test output. A package installation failure is different from a failed calculator assertion. GitHub's run logs show the steps and their output. (8)

The equivalent CLI commands are:

```
gh run list --workflow webcalc-tests.yml
gh run watch RUN_ID --exit-status
```

Replace `RUN_ID` with the run identifier from the list. The workflow must have been pushed and Actions must be allowed for the repository. No website deployment is involved; the test app runs temporarily inside the browser job.

Break a check and inspect the failed run

Now deliberately change the expected result in `test_calculator.py`. Find `test_addition` and change only its final assertion:

```
assert calc.display == '6'
```

The calculation is still `2 + 3`. This expectation is intentionally wrong. We are testing our feedback mechanism, not changing the product's arithmetic requirement.

Run the failing test locally so its output is familiar:

```
python -m pytest -v test_calculator.py -k test_addition
```

Expect one failure. The assertion compares the actual display `5` with the deliberately incorrect expected value `6`. Commit and push this known failure as part of the sandbox exercise:

```
git add test_calculator.py
git commit -m "Demonstrate a failing addition check"
git push
```

Open the new Actions run. Expect the unit job to fail while the simulated GUI and browser jobs pass. Their expectations have not changed. Open the failed unit step and find the same assertion seen locally. The test process returns a nonzero exit status, which makes that step and job fail.

This is also a useful reminder: a red test does not automatically mean the implementation is wrong. Here the test is wrong. We need to compare both against the agreed behavior.

Fix it and get a new passing run

Restore the assertion to:

```
assert calc.display == '5'
```

Run all three local layers again:

```
python -m pytest -v test_calculator.py
python -m pytest -v test_main.py
python run_browser_tests.py
```

Then commit and push the repair:

```
git add test_calculator.py
git commit -m "Restore the correct addition expectation"
git push
```

All three jobs should pass on the new run. The earlier failed run remains in the history. Its result describes the earlier revision; the new result describes the repaired one. Check the commit associated with a run before treating its status as evidence about your current code.

Make failures useful

A reliable process takes repetitive work out of people's heads. Nobody should have to remember which colleague last ran which test. The workflow records the commands and the results, so a team can discuss a concrete failure instead of wondering whether the shared version was checked.

Keep that feedback useful. Investigate failures promptly. If a test fails intermittently, find out whether the problem is in the product, the test, or the environment. Repeatedly rerunning until a green result appears leaves the uncertainty in place.

The requirements file pins our direct Python dependencies. It does not freeze every transitive dependency, the runner image, or the browser version. A later environment change can still affect a run. That is a reason to inspect logs and maintain the setup, not to dismiss automation.

Our browser tests use a local calculator instead of a public website. This keeps an unrelated service outage or content change out of the exercise. The jobs still need internet access to obtain code and packages. A project that requires credentials should use the platform's secret storage rather than put credentials in committed files.

Passing these tests gives us evidence about the cases we chose. It does not establish every possible result, visual layout, or user interaction. It also does not approve or deploy a change. Version Control and GitHub Workflow will use these familiar results when accepting work. The later Continuous Integration and Continuous Delivery topic will extend the process toward preparing and delivering a release, rather than repeat this Actions introduction.

Considerations

1. The unit job fails while the other two pass. What would you inspect before deciding whether to change the app or the test?
2. A developer has green local results, but the hosted browser job fails during startup. Which differences between those environments could explain it?

3. The browser scenarios take much longer than the unit tests. How could the team keep feedback fast without quietly dropping useful checks?
4. A workflow exists, but developers integrate their work only once a month. Which part of continuous integration is still missing?

Practice

1. Create the standalone `webcalc` repository and complete the local-pass, hosted-pass, break, and fix sequence. Record the commit identifiers for the three hosted runs and the outcome of each job.
2. Open `.github/workflows/webcalc-tests.yml`. Identify the three triggers, the Python version, and the command used by each test layer.
3. Download the artifacts from a run. Locate a unit test report and the browser app log. Describe what each can tell you about a failure.
4. Add a meaningful test for an agreed calculator behavior. Run it locally, commit it, and inspect the corresponding Actions run. Update your expected test count.

References

1. Wikipedia contributors. “Continuous integration.” https://en.wikipedia.org/wiki/Continuous_integration
2. NiceGUI. “User.” Testing documentation. https://nicegui.io/documentation/user
3. Selenium. “Selenium Manager.” https://www.selenium.dev/documentation/selenium_manager/
4. GitHub. “Understanding GitHub Actions.” https://docs.github.com/en/actions/get-started/understand-github-actions
5. GitHub CLI. “gh repo create.” https://cli.github.com/manual/gh_repo_create
6. GitHub. “Running variations of jobs in a workflow.” https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/run-job-variations
7. GitHub. “Store and share data with workflow artifacts.” https://docs.github.com/en/actions/tutorials/store-and-share-data
8. GitHub. “Using workflow run logs.” https://docs.github.com/en/actions/how-tos/monitor-workflows/using-workflow-run-logs